

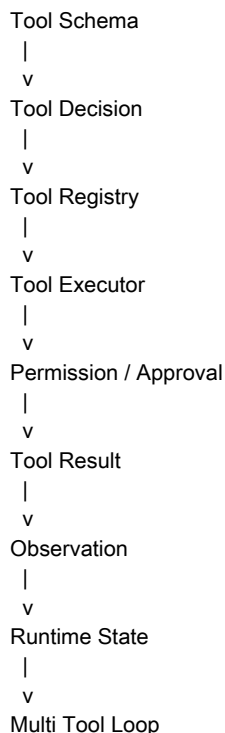
Day05 Part I 学习文档 v1.0 : Mini Tool Runtime Implementation (最小执行引擎实现)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part I 正式学习文档。

Day05 Part A 建立 Tool Calling 基础模型，Part B 解释 LLM 如何做 Tool Decision，Part C 说明 Tool Schema 如何定义行动契约，Part D 说明 Tool Registry 如何管理能力空间，Part E 说明 Tool Executor 如何托管执行，Part F 引入 Permission 与 Human Approval 治理层，Part G 说明 Tool Result 如何转成 Observation 回流 Runtime，Part H 说明 Multi Tool Loop 如何形成持续执行。Part I 开始把这些抽象收束成一个最小可运行的 Mini Tool Runtime 设计。

本节定位

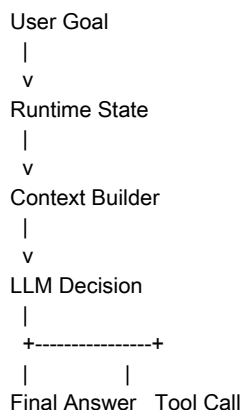
Part A-H 已经把 Execution Engine 的核心概念拆开讲完：

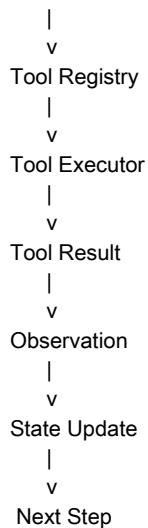


Part I 要回答的问题是：

这些概念在代码里如何真正连起来，形成一个最小可运行的 Agent Runtime？

这一节不是为了实现生产级 Agent 框架，而是做一次最小闭环验证：





本节核心结论是：

Mini Tool Runtime 的目标不是把 Tool 调起来，而是验证 Runtime 能否管理“决策 -> 执行 -> 反馈 -> 状态演化”的完整闭环。真正让 LLM + Tool 变成 Agent 的，不是某个函数调用，而是 Runtime 对 State、Context、Tool、Observation、Step 和 Guard 的统一调度。

目录

Part I 的目标

Mini Runtime 的最小闭环

Runtime Skeleton

RuntimeState

State 与 Context 的分离

AgentRuntime 的角色

Runtime 组合能力，而不是继承能力

run 生命周期

为什么不能只有 while(true)

Loop Guard

为什么第一版不急着抽 Planner

Tool System 在 Runtime 中的位置

Tool Interface

Tool Schema

Tool Registry

Registry 是能力边界

Tool Executor

为什么治理逻辑不应该放进 Tool

Permission 与 Approval 预留点

Observation Pipeline

Agent Executor Loop

StepRunner

AgentRuntime 与 StepRunner 的职责边界

Weather Agent Demo

完整 Trace

Day05 A-I 知识串联

OpenAI Agents SDK / LangGraph / Claude Code 映射

工业补充：Event System

工业补充：Reducer 与 State Transition Log

工业补充：Planner 抽象

工业补充：Action Layer

工业补充：Tool Result 可信度

工业补充：Multi-Agent

mini-agent-runtime 第一版实现路线

工业术语映射

面试视角

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

知识地图

思考题

前置问题回收

Part I 的目标

Part I 的目标不是从零写一个完整 SDK。

如果这一节一开始就追求：

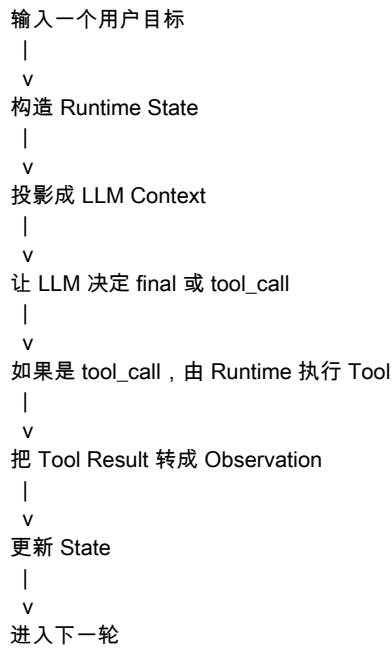
- 插件系统
- 持久化
- Streaming
- Human Approval UI
- Trace 平台
- Memory Retrieval
- MCP Client
- Workflow Graph

就会把 Day05 的核心目标冲散。

这一节只需要验证一件事：

Day05 Part A-H 的抽象是否真的能跑成一个最小 Runtime Loop。

所以 Mini Runtime 的第一版目标是：



这条链路一旦跑通，Day05 的 Execution Engine 就完成了闭环。

Mini Runtime 的最小闭环

从实现角度看，最少需要这些对象：

RuntimeState

ToolRegistry

ToolExecutor

ObservationProcessor

ContextBuilder

LLMClient

StepRunner

AgentRuntime

LoopGuard

可以先把它们分成三组：

State System
RuntimeState
Observation
ToolCallRecord

Execution System
Tool
ToolSchema
ToolRegistry

```
ToolExecutor
ObservationProcessor
```

```
Loop System
ContextBuilder
LLMClient
StepRunner
LoopGuard
AgentRuntime
```

这不是传统 Web 项目的 controller / service / dao 分层，而是 Agent Runtime 的概念分层。

关键不是类名多，而是每个类承载的边界不同：

- RuntimeState 描述当前运行现场。
- ContextBuilder 决定本轮给 LLM 看什么。
- LLMClient 返回下一步决策。
- ToolRegistry 管理 Runtime 当前能力。
- ToolExecutor 托管单个 Tool 的执行生命周期。
- ObservationProcessor 把外部结果转成 Runtime Feedback。
- StepRunner 跑一轮 Think / Act / Observe。
- AgentRuntime 调度多轮 Step。
- LoopGuard 控制停止条件和资源边界。

Runtime Skeleton

第一版目录可以这样设计：

```
src/
  runtime/
    AgentRuntime.ts
    RuntimeState.ts
    StepRunner.ts
    LoopGuard.ts

  context/
    ContextBuilder.ts

  llm/
    LLMClient.ts

  tools/
    Tool.ts
    ToolRegistry.ts
    ToolExecutor.ts

  observation/
    Observation.ts
    ObservationProcessor.ts

  index.ts
```

这个 Skeleton 的目的，是把 Day05 的抽象落成代码边界：

```
AgentRuntime
|
+-- StepRunner
|
```

```

+-- LoopGuard

StepRunner
|
+-- ContextBuilder
|
+-- LLMClient
|
+-- ToolExecutor
|
+-- ObservationProcessor

ToolExecutor
|
+-- ToolRegistry
|
+-- Tool

```

第一版可以很小，但边界要清楚。

否则很容易写成：

```

async function run(prompt) {
  const response = await llm.call(prompt);
  if (response.tool_call) {
    const result = await tools[response.tool_call.name](response.tool_call.arguments);
    return llm.call(String(result));
  }
  return response.content;
}

```

这段代码能跑 Demo，但它没有真正的 Runtime：

- 没有 State。
- 没有 Step。
- 没有 Observation。
- 没有 Guard。
- 没有 Trace。
- 没有恢复点。
- 没有清晰的执行边界。

Mini Runtime 的第一版不追求复杂，但必须保留这些概念的骨架。

RuntimeState

Part H 已经说明，Agent Loop 本质上是 State Transition Loop。

所以 Part I 首先要定义 State：

```

export interface RuntimeState {
  goal: string;
  messages: RuntimeMessage[];
  toolCalls: ToolCallRecord[];
  observations: Observation[];
  step: number;
  status:
    | "thinking"
    | "executing"
    | "waiting"
    | "completed"
}

```

```
    | "failed";  
  }
```

其中：

- goal 是当前任务目标。
- messages 保存用户和助手可追踪的对话事件。
- toolCalls 保存 Tool Call Chain。
- observations 保存外部世界反馈。
- step 标记当前执行轮次。
- status 描述 Runtime 当前阶段。

一个最小初始化状态：

```
const state: RuntimeState = {  
  goal: "查询北京天气",  
  messages: [  
    {  
      role: "user",  
      content: "北京今天天气怎么样？",  
    },  
  ],  
  toolCalls: [],  
  observations: [],  
  step: 0,  
  status: "thinking",  
};
```

这里有一个关键点：

用户输入不是直接丢给 LLM，而是先进入 Runtime State，再由 Context Builder 投影给 LLM。

这正是 Day04 的知识在 Day05 实现阶段的落地。

State 与 Context 的分离

Day04 已经建立过一个重要结论：

Runtime State != LLM Context

State 是 Runtime 当前世界。

Context 是 Runtime 本轮投影给 LLM 的视图。

例如 State 中可能保存完整结构：

```
{  
  "goal": "查询北京天气",  
  "step": 1,  
  "toolCalls": [  
    {  
      "id": "call_1",  
      "name": "get_weather",  
      "arguments": {  
        "city": "北京"  
      }  
    }  
  ],  
  "observations": [  
    {  
      "type": "tool_result",  
      "summary": "北京天气晴，26度",  
    }  
  ]  
}
```

```

    "payload": {
      "city": "北京",
      "weather": "晴",
      "temperature": 26
    }
  }
]
}

```

但 Context Builder 可能只投影成：

用户目标：查询北京天气

已有 Observation：
北京天气晴，26度。

请根据当前信息回答用户。

分离 State 与 Context 的好处是：

- Runtime 可以保留完整事实。
- LLM 只看到当前必要信息。
- Context 可以按 token budget 压缩。
- Observation 可以被排序、摘要、过滤。
- 后续支持 Memory、Checkpoint、Replay 更自然。

如果直接把 State 序列化塞给 LLM，Mini Demo 也许能跑，但后面会变成不可控的 Prompt 拼接。

AgentRuntime 的角色

很多初学实现会把 AgentRuntime 写成：

```

class AgentRuntime {
  tools = [];

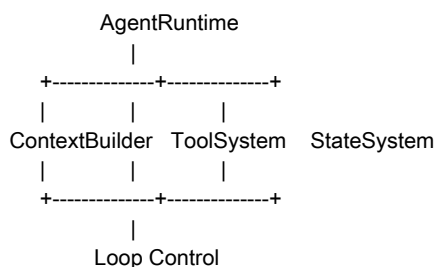
  async run(input: string) {
    return this.llm.call(input);
  }
}

```

这不是 Runtime。

这只是 LLM Wrapper。

真正的 Runtime 应该是 Coordinator：



AgentRuntime 不负责具体业务，也不应该知道某个天气 API 怎么调用。

它负责：

- 初始化运行现场。
- 调度 Step。

- 检查 Guard。
- 接收 StepResult。
- 更新状态。
- 决定继续或结束。
- 在未来发出 Runtime Event。

所以 AgentRuntime 的职责更像操作系统里的进程调度器，而不是某个业务函数。

Runtime 组合能力，而不是继承能力

Part I 中很重要的一个软件工程判断是：

Runtime 通过组合获得能力，而不是通过继承变成某个能力。

错误方向：

```
class GPTAgentRuntime extends ToolExecutor {}
```

这混淆了职责。

Runtime 不是 ToolExecutor。

Runtime 拥有 ToolExecutor。

更合理：

```
class AgentRuntime {
  constructor(
    private stepRunner: StepRunner,
    private guard: LoopGuard
  ) {}
}
```

再往下：

```
class StepRunner {
  constructor(
    private contextBuilder: ContextBuilder,
    private llm: LLMClient,
    private toolExecutor: ToolExecutor,
    private observationProcessor: ObservationProcessor
  ) {}
}
```

这种设计符合传统软件工程里的 Composition over Inheritance。

Agent Runtime 的很多设计，本质上不是神秘的新范式，而是传统工程原则在 LLM 执行系统里的重新落地。

run 生命周期

第一版 run 可以先理解为：

```
class AgentRuntime {
  constructor(
    private stepRunner: StepRunner,
    private guard: LoopGuard
  ) {}

  async run(state: RuntimeState): Promise<string> {
    while (true) {
      this.guard.check(state);
```

```

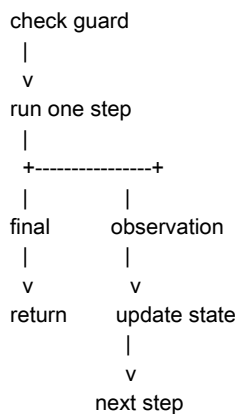
const result = await this.stepRunner.run(state);

if (result.type === "final") {
  state.status = "completed";
  return result.answer;
}

if (result.type === "observation") {
  state.observations.push(result.observation);
  state.step++;
  state.status = "thinking";
}
}
}
}

```

这段代码背后的执行语义是：



教学阶段可以直接修改 state，但要知道这是简化版。

工业版本一般会使用 Event + Reducer 生成新 State。

为什么不能只有 while(true)

裸 while(true) 的问题不是语法，而是缺少 Runtime 控制点。

例如：

```

while (true) {
  const decision = await llm.call(context);
  if (decision.final) break;
  await tool.execute(decision.toolCall);
}

```

它无法自然插入：

- max steps
- timeout
- token budget
- cost budget
- duplicate action detection
- no progress detection
- approval suspend
- trace event

- checkpoint
- resume

工业 Runtime 不会让循环自己无上限运行。

循环必须被 Runtime Policy 管住：

```
AgentRuntime
|
+-- LoopGuard
|
+-- StepRunner
```

所以第一版虽然仍然可能写 while，但语义上它不是裸循环，而是：

Guarded Execution Loop

Loop Guard

LoopGuard 是 Runtime 对 LLM 的资源边界控制。

最小接口：

```
export interface LoopGuard {
  check(state: RuntimeState): void;
}
```

最小实现：

```
export class MaxStepGuard implements LoopGuard {
  constructor(private maxSteps: number) {}

  check(state: RuntimeState): void {
    if (state.step >= this.maxSteps) {
      throw new Error("max step exceeded");
    }
  }
}
```

为什么必须由 Runtime 控制？

因为 LLM 可以提出：

再搜索一次。

但 Runtime 必须能说：

已经搜索 20 次，停止。

停止条件不能完全交给 LLM。LLM 负责推理下一步，Runtime 负责资源、风险和生命周期边界。

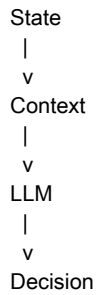
为什么第一版不急着抽 Planner

工业框架中经常会看到 Planner：

```
Planner
|
v
Action
|
v
Executor
```

但 Mini Runtime 第一版可以暂时不抽。

原因是现在：



LLM 本身已经承担了最小 Planner 的角色。

未来再抽象：

```
export interface Planner {  
    decide(context: RuntimeContext): Promise<Action>;  
}
```

然后支持：

```
LLMPlanner  
WorkflowPlanner  
HybridPlanner
```

这也连接到 Part H 里的结论：

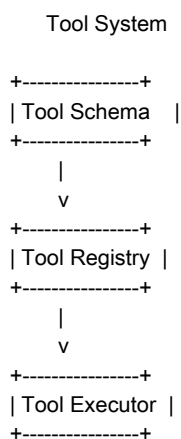
- 开放任务适合 LLM Planner。
- 固定业务流程适合 Workflow Planner。
- 很多企业 Agent 实际是 AI Assistant + Workflow。

第一版先让 LLM 直接输出 final 或 tool_call，是合理的最小实现。

Tool System 在 Runtime 中的位置

Tool System 不是一个工具数组。

它至少包含三层：



这三层分别解决不同问题：

- Tool Schema：LLM 如何理解能力。
- Tool Registry：Runtime 当前有哪些能力。
- Tool Executor：Runtime 如何安全可靠地执行能力。

不要把它们压成一个 tools 对象。

否则后续 Permission、Tool Routing、Dynamic Tool、MCP、Trace 都会变得别扭。

Tool Interface

最小 Tool 可以这样定义：

```
export interface Tool {  
  schema: ToolSchema;  
  execute(args: unknown): Promise<unknown>;  
}
```

其中 execute 是真实执行函数，schema 是给 LLM 和 Runtime 使用的能力契约。

天气 Tool：

```
const weatherTool: Tool = {  
  schema: {  
    name: "get_weather",  
    description: "查询指定城市天气",  
    parameters: {  
      type: "object",  
      properties: {  
        city: {  
          type: "string",  
        },  
      },  
      required: ["city"],  
    },  
  },  
  
  async execute(args) {  
    const { city } = args as { city: string };  
  
    return {  
      city,  
      weather: "晴",  
      temperature: 26,  
    };  
  },  
};
```

这里要注意：

Tool 不只是函数，Tool 是 Runtime 可调度的一项能力。

函数只解决执行问题。

Tool 还要解决：

- name
- description
- parameters
- metadata
- risk level
- permission policy
- output semantics

Mini Runtime 第一版可以只实现前三个，但心里要知道后面会扩展。

Tool Schema

Tool Schema 是 Part C 的核心落地。

它不是普通 API 文档，而是 LLM 与 Runtime 之间的行动契约：

```
export interface ToolSchema {  
  name: string;  
  description: string;  
  parameters: JSONSchema;  
}
```

LLM 看到的是 Tool Schema，而不是 Tool 的源码。

所以 Schema 的质量会直接影响：

- LLM 是否选择这个 Tool。
- 参数是否填对。
- Tool Call 是否稳定。
- Tool Result 是否能被下一轮正确理解。

如果 Schema 写成：

```
do something
```

LLM 就会很难判断何时调用。

如果 Schema 写成：

```
查询指定城市的实时天气。输入必须包含 city，返回 weather 与 temperature。
```

模型的行动空间就清晰得多。

Tool Registry

Tool Registry 的最小职责：

注册 Tool。

根据 name 查询 Tool。

暴露当前可用 Tool Schema 给 LLM。

最小实现：

```
export class ToolRegistry {  
  private tools = new Map<string, Tool>();  
  
  register(tool: Tool): void {  
    this.tools.set(tool.schema.name, tool);  
  }  
  
  get(name: string): Tool | undefined {  
    return this.tools.get(name);  
  }  
  
  getSchemas(): ToolSchema[] {  
    return [...this.tools.values()].map((tool) => tool.schema);  
  }  
}
```

其中 getSchemas() 非常关键。

因为 Context Builder 构造 LLM Context 时，需要知道当前应该暴露哪些能力：

```
const context = {  
  messages: state.messages,  
  observations: state.observations,  
  tools: registry.getSchemas(),  
};
```

Registry 管理能力空间。

Context Builder 管理信息空间。

二者共同决定 LLM 本轮能看到什么、能做什么。

Registry 是能力边界

Registry 不是简单的 Map<string, Tool>。

它实际上定义了 Agent 当前行动空间。

例如用户说：

删除数据库所有数据。

如果当前 Registry 没有：

delete_database

那么 LLM 在本轮 Tool Definition 中根本看不到这个能力。

这就是能力边界：

```
Runtime 所注册和暴露的 Tool  
|  
v  
LLM 可选择的 Action Space
```

生产系统里，Registry 还会继续扩展：

- 按用户权限过滤 Tool。
- 按任务类型路由 Tool。
- 按风险等级隐藏 Tool。
- 按版本选择 Tool。
- 从 MCP 或 Plugin 动态发现 Tool。
- 根据上下文只暴露最相关 Tool。

Mini Runtime 第一版可以只做 Map，但文档里要明确：Registry 的本质是 Capability Management。

Tool Executor

Tool Executor 负责把 Tool Call Intent 变成真实执行。

最小流程：

```
ToolCall  
|  
v  
ToolRegistry.get(name)  
|  
v
```

```
Tool.execute(arguments)
|
v
ToolResult
```

最小实现：

```
export class ToolExecutor {
  constructor(private registry: ToolRegistry) {}

  async execute(toolCall: ToolCall): Promise<unknown> {
    const tool = this.registry.get(toolCall.name);

    if (!tool) {
      throw new Error(`Tool not found: ${toolCall.name}`);
    }

    return tool.execute(toolCall.arguments);
  }
}
```

这只是第一版。

Part E 已经说明，生产级 Tool Executor 还需要托管：

- input validation
- business validation
- permission check
- human approval
- timeout
- retry
- cancellation
- idempotency
- audit log
- tracing
- error mapping

所以 Tool Executor 的定位是：

Agent Runtime 的 Execution Kernel。

不是普通函数调用器。

为什么治理逻辑不应该放进 Tool

一个常见错误是：

```
const refundTool = {
  async execute(args) {
    validate(args);
    checkPermission(args);
    retry(async () => callRefundApi(args));
    log(args);
  },
};
```

这样每个 Tool 都会重复实现：

- 参数校验。
- 权限检查。
- 超时。
- 重试。
- 日志。
- 审计。

最后 Tool 既是业务能力，又是 Runtime Governance，职责混乱。

正确边界：

```
Tool
|
+-- 描述业务能力
+-- 执行最小业务动作
```

```
ToolExecutor
|
+-- 校验
+-- 权限
+-- 审批
+-- 超时
+-- 重试
+-- Trace
+-- Error Mapping
```

Tool 负责能力。

Executor 负责治理。

这就是 Part E 在代码里的核心意义。

Permission 与 Approval 预留点

Mini Runtime 第一版可以不实现完整审批系统，但接口位置要想清楚。

执行前：

```
ToolCall
|
v
Input Validation
|
v
Permission Check
|
+-- allow -> execute
|
+-- deny -> observation(error)
|
+-- approval_required -> suspend
```

这说明 Permission 不应该是 Tool 里的某个 if。

它应该是 Tool Executor Pipeline 中的治理节点。

例如未来可以扩展：

```
export class ToolExecutor {
  constructor(
    private registry: ToolRegistry,
    private permissionService: PermissionService
```

```

) {}

async execute(toolCall: ToolCall): Promise<ToolExecutionResult> {
  const permission = await this.permissionService.check(toolCall);

  if (permission.decision === "deny") {
    return {
      type: "denied",
      reason: permission.reason,
    };
  }

  if (permission.decision === "approval_required") {
    return {
      type: "approval_required",
      request: permission.request,
    };
  }

  const tool = this.registry.get(toolCall.name);
  if (!tool) {
    return {
      type: "error",
      error: `Tool not found: ${toolCall.name}`,
    };
  }

  return {
    type: "success",
    result: await tool.execute(toolCall.arguments),
  };
}
}

```

第一版不需要写满，但边界要为 Part F 留出位置。

Observation Pipeline

Part G 的核心结论是：

Tool Result 不是 Final Answer，也不应该直接塞回 LLM。

正确链路：

```

Tool Result
|
v
Observation Processor
|
v
Observation
|
v
Runtime State
|
v
Context Builder
|
v
LLM

```

最小 Observation：

```

export interface Observation {
  id: string;
  type: "tool_result" | "tool_error" | "approval_result";
}

```

```
summary: string;
payload: unknown;
}
```

最小 Processor :

```
export class ObservationProcessor {
  process(result: unknown): Observation {
    return {
      id: crypto.randomUUID(),
      type: "tool_result",
      summary: JSON.stringify(result),
      payload: result,
    };
  }
}
```

后续可以扩展 :

- Validator
- Normalizer
- Summarizer
- Priority
- Source
- Confidence
- Token cost
- Visibility policy

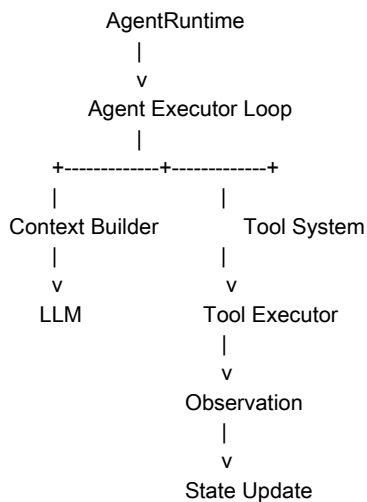
Observation 是 Runtime 理解外部世界的统一反馈事件。

Agent Executor Loop

Part H 说明 , Agent 不是一次 Tool Call , 而是多步循环。

Part I 要把循环实现出来。

完整结构 :



这里要区分两个 Executor :

Agent Executor
管整个任务循环

Tool Executor
管单个 Tool 执行

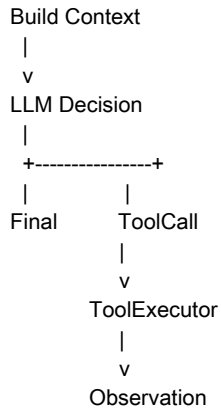
不要把它们混成一个类。

否则单步执行治理和多步任务调度会互相污染。

StepRunner

工业 Runtime 常把每一轮 Agent Loop 称为 Step。

一个 Step 的流程是：



接口：

```
export interface StepRunner {
  run(state: RuntimeState): Promise<StepResult>;
}
```

StepResult：

```
export type StepResult =
| {
  type: "final";
  answer: string;
}
| {
  type: "observation";
  observation: Observation;
};
```

实现：

```
export class DefaultStepRunner implements StepRunner {
  constructor(
    private contextBuilder: ContextBuilder,
    private llm: LLMClient,
    private toolExecutor: ToolExecutor,
    private observationProcessor: ObservationProcessor
  ) {}

  async run(state: RuntimeState): Promise<StepResult> {
    const context = this.contextBuilder.build(state);
    const decision = await this.llm.run(context);

    if (decision.type === "final") {
      return {
        type: "final",
        answer: decision.answer,
      };
    }
  }
}
```

```

    if (decision.type === "tool_call") {
      const result = await this.toolExecutor.execute(decision.toolCall);
      const observation = this.observationProcessor.process(result);

      return {
        type: "observation",
        observation,
      };
    }

    throw new Error(`Unknown decision type`);
  }
}

```

StepRunner 的价值是：

- 让“一轮怎么跑”独立出来。
- 让 AgentRuntime 只负责“是否继续跑”。
- 方便插入 Trace、Streaming Event、Retry、Approval。
- 方便测试单步行为。

AgentRuntime 与 StepRunner 的职责边界

边界可以这样记：

```

StepRunner
|
+-- 本轮如何思考
+-- 本轮如何执行
+-- 本轮产生什么结果

AgentRuntime
|
+-- 是否开始
+-- 是否继续
+-- 是否停止
+-- 如何推进 state.step
+-- 如何处理 final / failed / waiting

```

这使得 AgentRuntime 不需要知道 Tool 的细节。

它只看 StepResult：

```

final -> 完成

observation -> 更新 State，进入下一轮

error -> 失败或恢复策略

waiting -> 暂停，等待 Human Approval

```

未来扩展 Human Approval 时，可以让 StepResult 增加：

```

{
  type: "waiting";
  approvalRequest: ApprovalRequest;
}

```

AgentRuntime 收到后把 state.status 设为 "waiting"，并保存 Resume Token。

这就是 Part F 与 Part H 在实现层的接口位置。

Weather Agent Demo

用天气查询验证 Mini Runtime。

用户输入：

北京今天天气怎么样？

注册 Tool：

```
registry.register({
  schema: {
    name: "get_weather",
    description: "查询城市天气",
    parameters: {
      type: "object",
      properties: {
        city: {
          type: "string",
        },
      },
      required: ["city"],
    },
  },

  async execute(args) {
    const { city } = args as { city: string };

    return {
      city,
      weather: "晴",
      temperature: 26,
    };
  },
});
```

初始化 State：

```
const state: RuntimeState = {
  goal: "查询北京天气",
  messages: [
    {
      role: "user",
      content: "北京今天天气怎么样？",
    },
  ],
  toolCalls: [],
  observations: [],
  step: 0,
  status: "thinking",
};
```

第一轮 Context：

```
{
  "messages": [
    {
      "role": "user",
      "content": "北京今天天气怎么样？"
    }
  ],
  "tools": [
    {
      "name": "get_weather",
      "description": "查询城市天气"
    }
  ]
}
```

```
}
```

LLM Decision :

```
{
  "type": "tool_call",
  "toolCall": {
    "name": "get_weather",
    "arguments": {
      "city": "北京"
    }
  }
}
```

Runtime 执行 :

```
ToolRegistry.get("get_weather")
|
v
ToolExecutor.execute()
|
v
weatherTool.execute({ city: "北京" })
```

Tool Result :

```
{
  "city": "北京",
  "weather": "晴",
  "temperature": 26
}
```

Observation :

```
{
  "type": "tool_result",
  "summary": "北京天气晴，26度",
  "payload": {
    "city": "北京",
    "weather": "晴",
    "temperature": 26
  }
}
```

State Update :

```
{
  "step": 1,
  "observations": [
    {
      "type": "tool_result",
      "summary": "北京天气晴，26度"
    }
  ],
  "status": "thinking"
}
```

第二轮 Context :

用户问题 :
北京今天天气怎么样？

Observation :
北京天气晴，26度。

LLM Final :

```
{
  "type": "final",
}
```

```
"answer": "北京今天晴，26度。"  
}
```

Runtime 返回最终答案。

完整 Trace

如果打印执行轨迹，应该类似：

```
[Step 0]  
User:  
北京今天天气怎么样？
```

```
[Decision]  
ToolCall:  
get_weather
```

```
[Execution]  
Tool:  
get_weather
```

```
Args:  
{  
  city: "北京"  
}
```

```
[Observation]  
{  
  weather: "晴",  
  temperature: 26  
}
```

```
[Step 1]
```

```
[Decision]  
Final Answer
```

```
[Result]  
北京今天晴，26度。
```

这个 Trace 已经接近工业 Agent 的执行轨迹。

OpenAI Agents SDK、LangGraph、Claude Code 等框架虽然实现细节不同，但都会保留类似的执行事件：

- LLM start
- LLM decision
- tool called
- tool completed
- observation added
- state updated
- final answer

这说明 Trace 不是调试时临时打印的日志，而是 Runtime 的一等输出。

Day05 A-I 知识串联

Day05 现在可以连成一条完整链路。

Part A : Tool Calling 基础

LLM 可以输出结构化 Tool Call Intent。

Part B : Tool Decision

Tool Calling 不是 if/else , 而是 Goal-driven Action Selection。

Part C : Tool Schema

Tool Schema 是 LLM 与 Runtime 的行动契约。

Part D : Tool Registry

Tool Registry 管理 Agent 当前能力空间。

Part E : Tool Executor

Tool Executor 把 Tool Call Intent 转成安全、可靠、可观测的执行。

Part F : Permission & Human Approval

Runtime Governance 限制 Agent 代表用户采取行动。

Part G : Observation Feedback

Tool Result 转成 Observation 后进入 Runtime State。

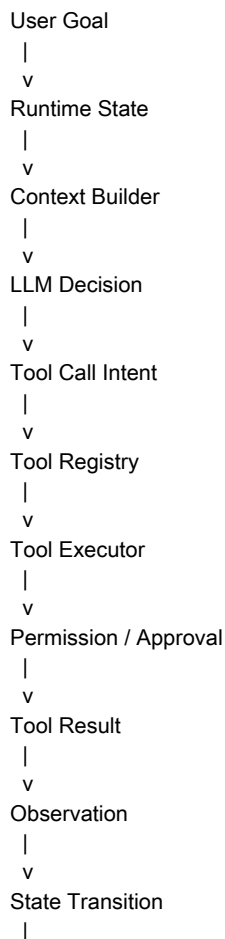
Part H : Multi Tool Loop

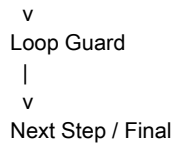
Agent 在 Think / Act / Observe / Update State 中持续完成目标。

Part I : Mini Tool Runtime Implementation

把 State、Context、Decision、Tool、Observation、Loop 和 Guard 落成最小可运行设计。

所以 Day05 的最终链路是：

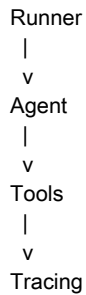




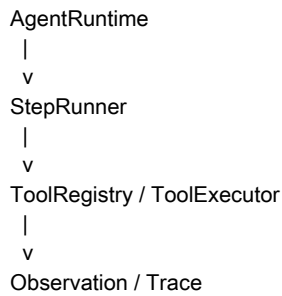
OpenAI Agents SDK / LangGraph / Claude Code 映射

Mini Runtime 与工业框架的概念可以这样对应。

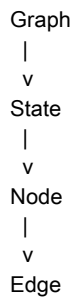
OpenAI Agents SDK :



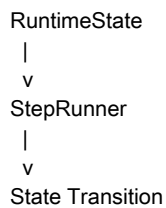
对应 Mini Runtime :



LangGraph :



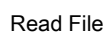
对应 Mini Runtime :

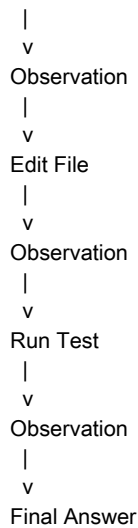


区别是 :

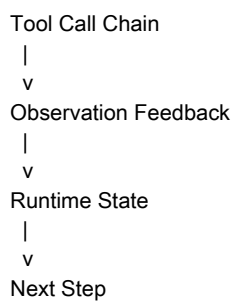
- LangGraph 更强调显式图和确定路径。
- Mini Agent Loop 更强调动态下一步决策。

Claude Code :





对应 Mini Runtime :



Coding Agent 是当前最容易看懂 Agent Runtime 的成熟场景，因为它天然具备：

- 多步执行。
- 工具调用。
- 文件和命令反馈。
- 错误恢复。
- Trace。
- Human-in-the-loop。

工业补充：Event System

Mini Runtime 教学阶段可以直接返回结果。

但工业 Runtime 一般会产生 Runtime Event：

```
export type RuntimeEvent =  
  | {  
    type: "llm_start";  
  }  
  | {  
    type: "llm_completed";  
  }  
  | {  
    type: "tool_called";  
    tool: string;  
    arguments: unknown;  
  }  
  | {  
    type: "tool_completed";
```

```

    tool: string;
    result: unknown;
  }
| {
  type: "observation_added";
  observation: Observation;
}
| {
  type: "state_updated";
}
| {
  type: "final";
  answer: string;
};

```

Event Stream 可以被多个系统消费：

```

Runtime
|
v
Event Stream
|
+-- UI
|
+-- Trace
|
+-- Logging
|
+-- Monitoring

```

Claude Code 之所以能实时显示：

```

Reading file
Editing file
Running tests

```

本质上就是 Runtime 暴露了 Execution Event Stream。

所以 Day07 Streaming Event 会自然接上 Day05。

工业补充：Reducer 与 State Transition Log

Part I 的教学代码里可能写：

```

state.step++;
state.observations.push(observation);

```

这可以帮助理解，但不是工业最佳形态。

工业 Runtime 更可能采用：

```

Event
|
v
Reducer
|
v
New State

```

类似 Redux：

```

function reducer(
  state: RuntimeState,
  event: RuntimeEvent
): RuntimeState {
  switch (event.type) {
    case "observation_added":

```

```

return {
  ...state,
  observations: [
    ...state.observations,
    event.observation,
  ],
  step: state.step + 1,
};

default:
  return state;
}
}

```

这样做是为了：

- Replay：重新播放执行过程。
- Debug：定位某个决策发生前的 State。
- Checkpoint：保存关键状态。
- Audit：审计 Agent 做过什么。
- Recovery：失败后从某个状态恢复。

这也连接到 Day04 的 Runtime State Lifecycle：

工业系统不只是保存当前对象，而是保存状态迁移历史。

工业补充：Planner 抽象

Mini Runtime 第一版可以让 LLM 直接决策：

```
const decision = await llm.run(context);
```

但工业系统通常会抽象 Planner：

```

export interface Planner {
  decide(context: RuntimeContext): Promise<Action>;
}

```

然后不同场景使用不同 Planner：

LLMPlanner
适合开放任务

WorkflowPlanner
适合固定业务流程

HybridPlanner
适合流程固定但局部需要模型判断

这解释了为什么很多企业 Agent 更接近：

AI Assistant + Workflow

因为企业任务往往有固定审批流、固定表单、固定业务规则，不一定适合让 LLM 自由规划每一步。

真正的架构判断不是“Agent 更高级，Workflow 更低级”，而是：

路径确定 -> Workflow

路径不确定 -> Agent Loop

路径部分确定 -> Hybrid Planner

工业补充：Action Layer

现在链路是：

```
LLM
|
v
ToolCall
|
v
ToolExecutor
```

工业系统中间可能还有 Action Layer：

```
LLM
|
v
Action
|
v
Policy Check
|
v
ToolExecutor
```

Action Layer 的作用是把模型输出的意图再抽象一层：

```
{
  "action": "refund",
  "amount": 10000
}
```

Runtime 可以先根据 Action 做治理：

- 这类 Action 是否允许？
- 金额是否超过阈值？
- 是否需要人工审批？
- 是否需要二次确认？
- 是否能映射到一个或多个 Tool？

然后再进入 ToolExecutor。

这可以看作 Part F Human Approval 的延伸。

工业补充：Tool Result 可信度

很多教程默认：

```
Tool Result
|
v
Observation
|
v
LLM
```

但工业场景还需要处理 Tool Result 可信度。

例如 Tool 返回：

```
{
  "status": "SUCCESS",
```

```
"balance": -100000000
}
```

Runtime 不能无条件相信并投影给 LLM。

Observation Processor 后面可能还需要：

```
Validator
|
v
Normalizer
|
v
Summarizer
|
v
Observation
```

也就是说：

Tool Result 是外部世界返回的数据。
Observation 是 Runtime 验证、归一化、摘要后的反馈事件。

这进一步强化 Part G 的结论：Observation 不是简单数据清洗层，而是 Runtime 与外部世界之间的隔离层。

工业补充：Multi-Agent

提前埋一个伏笔：

Multi-Agent 不只是多个 LLM，而是多个 Runtime Instance 加通信协议。

更准确：

```
Agent A Runtime
|
+-- State
+-- Context
+-- Tools
+-- Loop

Agent B Runtime
|
+-- State
+-- Context
+-- Tools
+-- Loop

Agent Communication
|
v
Message / Event / Handoff
```

例如：

```
Planner Runtime
|
v
Coder Runtime
|
v
Reviewer Runtime
```

每个 Agent 都有自己的 State、Context、Tools 和 Loop。

所以 Multi-Agent 的复杂度不只是“多个模型互相聊天”，而是多个 Runtime 的协调、隔离、通信和治理。

mini-agent-runtime 第一版实现路线

如果后续进入代码实现，可以按这个顺序：

1. State 类型

RuntimeState
RuntimeMessage
ToolCall
ToolCallRecord
Observation
StepResult

2. Tool System

ToolSchema
Tool
ToolRegistry
ToolExecutor

3. Context Builder

RuntimeState -> RuntimeContext

第一版只需要：

- messages
- observations
- tool schemas

4. LLMClient

第一版可以先用 Mock LLM：

Step 0 -> tool_call get_weather
Step 1 -> final answer

这样可以先测试 Runtime，而不依赖真实模型。

5. StepRunner

实现一轮：

build context -> llm decision -> execute tool -> observation

6. AgentRuntime

实现多轮：

guard -> step -> update state -> next step / final

7. Weather Demo

验证：

User
|
v
get_weather
|
v
Observation
|
v
Final Answer

这个路线能保证代码是从 Runtime 概念长出来的，而不是从某个 SDK 调用样例长出来的。

工业术语映射

| Mini Runtime 概念 | 工业常见名称 | 职责 | | --- | --- | --- | | AgentRuntime | Agent Runner / Runtime Runner | 调度整个任务生命周期 | | RuntimeState | Agent State / Execution State | 保存当前运行现场 | | ContextBuilder | Context Management / Context Engineering | 将 State 投影给 LLM | | LLMClient | Model Adapter / Model Client | 调用模型并返回决策 | | Planner | Planner / Policy / Decider | 选择下一步 Action | | ToolSchema | Function Schema / Capability Schema | 描述能力契约 | | ToolRegistry | Capability Registry / Tool Registry | 管理可用能力 | | ToolExecutor | Tool Runtime / Execution Kernel | 托管单个 Tool 执行 | | ObservationProcessor | Feedback Adapter / Result Processor | 结果转 Observation | | Observation | Observation / Execution Feedback | 外部世界反馈事件 | | StepRunner | Execution Step / Step Runner | 执行一轮 Agent Step | | LoopGuard | Runtime Policy / Guardrail | 控制循环和资源边界 | | RuntimeEvent | Trace Event / Execution Event | 对外暴露执行过程 | | Reducer | State Reducer / State Transition | 根据事件生成新 State |

面试视角

Q1：Agent 和普通 Chatbot 最大区别是什么？

普通 Chatbot 通常是一次 LLM 调用：

User -> LLM -> Answer

Agent 是 Runtime 驱动的循环执行系统：

State -> Context -> Decision -> Action -> Observation -> State

区别不是有没有接 API，而是有没有 Runtime 管理状态、工具、反馈和循环。

Q2：Tool Calling 的核心是什么？

Tool Calling 不是模型直接调用函数。

更准确：

LLM 产生 Tool Call Intent

Runtime 验证、治理、执行 Tool

Tool Result 转成 Observation

Observation 回流 Runtime State

Tool Calling 的核心是 Runtime 托管的行动闭环。

Q3：为什么 Agent 需要 Runtime？

因为 LLM 只负责生成下一步决策，不负责：

- 状态管理
- Context 投影
- 权限控制
- Tool 执行
- 结果验证

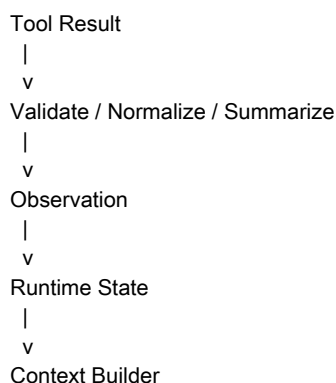
- 循环停止
- Trace
- 恢复

这些都是 Runtime 的职责。

Q4：为什么 Tool Result 不能直接作为下一轮 Prompt？

因为 Tool Result 是外部系统原始输出，不一定适合直接给 LLM。

它需要先转成 Observation：



这样 Runtime 才能控制反馈质量、可信度、优先级和 token 成本。

Q5：为什么 while(true) 不是工业 Agent Loop？

因为工业 Agent Loop 需要：

- Step
- State Transition
- Loop Guard
- Stop Condition
- Trace
- Approval Suspend / Resume
- Cost Control
- Error Recovery

裸循环只是控制流语法，不是 Runtime 设计。

下一节学习计划

Day05 Execution Engine 到 Part I 已经完成闭环。

下一节进入：

Day06：Memory System

核心问题：

Agent 如何从一次性执行系统，升级成拥有长期记忆的持续智能系统？

重点学习：

Conversation 为什么不等于 Memory

Short-term Memory 与 Long-term Memory

Memory Extraction

Memory Retrieval

Memory Injection into Context

Memory 与 Runtime State 的关系

Memory 与 Context Builder 的关系

Memory 与 Observation 的关系

OpenAI Agents SDK / Claude Code / LangGraph 中 Memory 如何实现

mini-agent-runtime 中如何实现最小 Memory System

写书 TODO

未来写 Day05 Execution Engine 章节时，可以把 Part I 拆成：

5.x Mini Agent Runtime Implementation

5.x.1 Runtime Skeleton

5.x.2 RuntimeState 与 ContextBuilder

5.x.3 Tool System : Schema / Registry / Executor

5.x.4 Observation Pipeline

5.x.5 Agent Executor Loop

5.x.6 StepRunner 与 LoopGuard

5.x.7 Weather Agent Demo

5.x.8 从 Mini Runtime 到工业 Runtime

需要补充的问题：

Mini Runtime 的代码实现是否应该先使用 Mock LLM？

Tool Schema 是否直接采用 JSON Schema 子集？

第一版 Observation 是否需要区分 summary 和 payload？

State 更新第一版是否直接 mutation，第二版再引入 Reducer？

Event System 应该在 Day05 实现，还是留到 Day07 Streaming Event？

Planner 抽象是否在 Day05 预留接口，Day09 Workflow 再展开？

Weather Demo 是否足够，是否再补一个连续两次 Tool Call 的 Demo？

如何把 Part I 的代码与后续 Day06 Memory System 衔接？

写书素材

素材 1：AgentRuntime 不是 Agent 本身

AgentRuntime 不是 Agent 本身。

它是 Agent 生命周期管理器。

这个句子适合放在 Runtime Skeleton 小节开头。

素材 2：Tool 不是函数

Tool 不是函数。

Tool 是 Runtime 可调度的一项能力契约。

这个句子适合放在 Tool Interface 小节。

素材 3：Observation 不是 Result

Observation 不是 Tool Result。

Observation 是 Runtime 对外部世界反馈的理解。

这个句子适合放在 Observation Pipeline 小节。

素材 4：Agent 是 Runtime 驱动的状态机

Agent = Runtime-driven State Machine

展开：

```
State
|
v
Context
|
v
Decision
|
v
Action
|
v
Observation
|
v
State Transition
```

素材 5：Tool 不是能力全部

给 Agent 加 API，不等于让 Agent 变聪明。

真正决定 Agent 能力的是 Runtime 如何组织 Context、Decision、Loop、Memory 和 Tool。

本 Part 核心认知升级

学习 Part I 前：

Agent = LLM + Tools

学习 Part I 后：

Agent = Runtime 驱动的状态机

更完整：

```
Agent
=
Runtime
```

- + LLM Decision
- + Tool Execution
- + Observation Feedback
- + State Transition
- + Control Policy

LLM 负责：

想下一步做什么

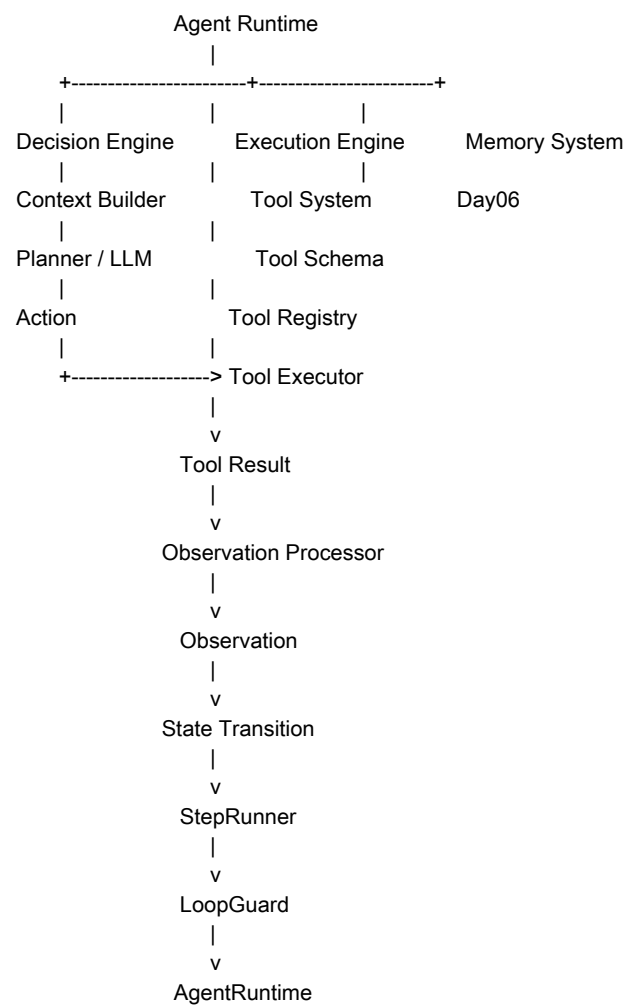
Runtime 负责：

- 能不能做
- 怎么做
- 做完如何记录
- 如何反馈给下一轮
- 什么时候停止

所以 Day05 最终完成的认知跃迁是：

Agent 的核心不是调用工具，而是 Runtime 如何管理“决策 -> 执行 -> 反馈 -> 状态演化”。

知识地图



思考题

为什么 Mini Runtime 第一版也必须有 RuntimeState ?

为什么 ContextBuilder 不应该直接读取用户输入字符串，而应该读取 RuntimeState ?

Tool Schema、Tool Registry、Tool Executor 分别解决什么问题？

为什么 ToolExecutor 不应该属于 Tool ?

为什么 Tool Result 需要先转成 Observation ?

StepRunner 和 AgentRuntime 的职责边界是什么？

为什么 Loop Guard 应该由 Runtime 控制，而不是交给 LLM ?

如果要支持 Human Approval，StepResult 应该如何扩展？

如果要支持 Streaming Event，AgentRuntime 应该在哪些节点发事件？

为什么 Multi-Agent 更准确地说多个 Runtime Instance，而不是多个 LLM ?

前置问题回收

Q1：Context Builder 为什么是 Runtime 核心？

因为完整链路是：

```
Runtime State
|
v
Context Builder
|
v
LLM Decision
|
v
Action
|
v
Observation
|
v
State Update
```

Context Builder 是 Runtime 与 LLM 的桥梁。

LLM 看到的不是 Runtime 本身，而是 Runtime 投影出来的当前任务视图。

Q2：Tool Calling 为什么不等于 Agent？

单次 Tool Calling 只是一次 Action。

Agent 需要：

```
Tool Calling
+
Observation
+
State Update
+
Loop Control
```

只有 Tool Calling + Loop 才接近真正 Agent。

Q3 : Mini Runtime 和生产 Runtime 的差距在哪里？

Mini Runtime 有核心执行骨架：

- State
- Context
- Tool
- Observation
- Step
- Guard

生产 Runtime 还需要：

- Persistence
- Checkpoint
- Event Stream
- Trace
- Approval
- Retry
- Cost Control
- Memory
- MCP / Plugin
- Workflow / Planner

Mini Runtime 不是终点，而是后续系统能力的地基。

参考

- ChatGPT 分享记录：<https://chatgpt.com/share/6a7546d5-9990-83e8-a066-012d543445d4>
- 本地源记录：source/day05-part-i-chatgpt-share-source.md